DEMO VIDEO SCRIPT & VIVA VOCE CHEAT SHEET

DocuQuery AI: Enterprise Retrieval-Augmented Generation Platform

Course: Java Project Based Learning (PBL) — 5th Semester
Institution: Arya College of Engineering & I.T., Jaipur (Dept. of AI&DS - Artificial Intelligence & Data Science)
Mentor: Er. Ram Babu Buri

PART 1: 10-MINUTE DEMO VIDEO RECORDING SCRIPT

Recording Setup

- **Screen resolution**: 1080p (1920x1080).
- **Software**: OBS Studio or Windows Game Bar (`Win + Alt + R`).
- **Visuals**: Split between the **DocuQuery AI Desktop App** (`demo_app.html` or Electron client), the **Neon Cloud PostgreSQL console**, and the **Spring Boot terminal**.

[0:00 – 1:30] Introduction & Problem Framing

Speaker: Shaurya Dixit (Team Leader)
Visual on Screen: Title Slide / DocuQuery AI desktop landing screen.
Spoken Script:

> *"Good morning respected evaluators, external examiner, and our project guide, Er. Ram Babu Buri sir. I am Shaurya Dixit, Team Leader for Project DocuQuery AI, along with my team members Yashvardhan, Vaibhav, Rajveer, and Yashpal from 5th Semester Artificial Intelligence & Data Science at Arya College of Engineering & I.T., Jaipur.*
>
> *When organizations use standard AI models like ChatGPT on private research papers or university syllabi, the models hallucinate—inventing facts and fabricating citations. To solve this, we built **DocuQuery AI**, an enterprise Retrieval-Augmented Generation platform built entirely on a **Java 21 Spring Boot 3** backend, **Neon Serverless PostgreSQL with pgvector**, and an **Electron cross-platform desktop frontend**, evaluating on the **Kaggle Academic Research Paper Corpus**.*
>
> *Our project is structured into 5 independent architectural modules. Let us demonstrate how a document moves through our pipeline in real time."*

[1:30 – 3:15] Module 2: Document Ingestion & Chunking Demo

Speaker: Yashvardhan Singh Chauhan
Visual on Screen: Click on the **Document Ingestion** tab in the desktop application. Drag and drop a multi-page PDF (`Cloud_Computing_Architecture.pdf` from Kaggle dataset).
Spoken Script:

> *"I am Yashvardhan Singh Chauhan, responsible for Module 2: Document Ingestion and Chunking.*

> *Here on the screen, I am uploading a complex 14-page PDF from our Kaggle academic corpus. As soon as I drop this file, our Spring Boot backend triggers an asynchronous worker powered by Apache Tika and PDFBox.*
>
> *Notice what happens: rather than splitting text blindly by character count, our recursive sliding-window chunker segments the document into 500-token chunks with a 50-token overlap, using Java's `BreakIterator` to preserve complete sentence boundaries. This PDF is parsed into exactly 42 semantic chunks, tagged with metadata, and prepared for vector indexing without blocking the user interface."*

[3:15 – 5:00] Module 3: Neon PostgreSQL Vector Storage & Security Demo

Speaker: Vaibhav Tank
Visual on Screen: Switch to the database terminal or Neon cloud console showing the `document_chunks` table and SQL query.
Spoken Script:

> *"I am Vaibhav Tank, responsible for Module 3: Database Architecture and Security.*
>
> *Instead of fragmenting our system with a separate external vector database, we utilize Neon Serverless PostgreSQL with the `pgvector` extension. Here in our database, each chunk is stored alongside its 1536-dimensional dense embedding vector.*
>
> *We have tuned a Hierarchical Navigable Small World (HNSW) index on `vector_cosine_ops`. When a query arrives, Neon executes an approximate nearest neighbor search using the cosine distance operator `<=>`, returning the Top-4 closest passages in under 94 milliseconds. Furthermore, all endpoints are secured with Spring Security 6 stateless JWT tokens and Bucket4j rate limiting to defend against denial-of-service attempts."*

[5:00 – 7:00] Module 1 & 4: Live RAG Chat, Streaming & Citations Demo

Speaker: Rajveer Singh Ranawat (supported by Shaurya Dixit)
Visual on Screen: Switch back to the **RAG Chat** tab. Type a natural language question: "*What is the core architecture for cloud service orchestration?*" Click **Ask RAG**.
Spoken Script:

> *"I am Rajveer Singh Ranawat, responsible for Module 4: React UI and Electron Desktop Shell. Now let us ask a technical question based on our uploaded document.*
>
> *Notice how the answer streams token-by-token using Server-Sent Events (SSE). Shaurya's Module 1 orchestrator retrieved the Top-4 chunks from Neon, assembled a strict hallucination-resistant prompt, and synthesized this grounded response.*
>
> *Most importantly, notice these blue **Citation Badges** at the bottom. When I click on `[Cloud_Computing_Architecture.pdf (p. 4)]`, an excerpt inspector modal opens immediately, showing the exact source sentence and similarity score. This provides 100% verifiable factual grounding."*

[7:00 – 8:45] Module 5: RAG Triad Evaluation & Admin Telemetry Demo

Speaker: Yashpal Singh Bhati
Visual on Screen: Switch to the **RAG Triad & Telemetry** tab.

Spoken Script:

> *"I am Yashpal Singh Bhati, responsible for Module 5: RAG Triad Evaluation and Testing.*
>
> *To objectively verify that our system eliminates hallucinations, we benchmarked the pipeline across 50 ground-truth query sets from the Kaggle dataset using the RAG Triad framework. Here on our live telemetry dashboard, you can see:*
> *1. **Context Relevance**: 88.6% of retrieved passages strictly match the query.*
> **2. **Faithfulness (Groundedness)**: 92.4% of generated statements are directly entailed by retrieved excerpts.**
> *3. **Average Retrieval Latency**: 94 milliseconds under our Apache JMeter load tests with 50 concurrent virtual users.*
>
> *Our backend test suite achieves 84.6% branch coverage using JUnit 5 and Mockito."*

[8:45 – 10:00] Conclusion & Project Wrap-Up

Speaker: Shaurya Dixit (Lead)
Visual on Screen: Show the GitHub repository with all 5 feature branches, commit history, and the tagged release `v1.0.0`.
Spoken Script:

> *"In conclusion, DocuQuery AI satisfies all Java PBL objectives by providing an end-to-end, high-performance RAG platform in Java Spring Boot, eliminating database sprawl through Neon `pgvector`, and delivering an intuitive desktop client with verifiable citations.*
>
> *All 5 team members have version-controlled their independent modules across our 5 GitHub feature branches with minimum 3 commits per student per week. We thank our guide Er. Ram Babu Buri sir and our department faculty. We are now open for your viva voce questions. Thank you!"*

PART 2: EXTERNAL VIVA VOCE Q&A CHEAT SHEET

Q1: What is the difference between Naive RAG and Modular RAG, and which one did you implement?

> ***Answer**:*
> *Naive RAG follows a rigid Retrieve-Read pipeline: blindly slicing text by character count, embedding, retrieving Top-K, and passing everything into the prompt. This causes the "lost in the middle" problem and high hallucination.*
> *Modular RAG (which our project implements) introduces intermediate optimization components:*
>
> 1. *Token-aware recursive chunking with sentence boundary preservation (`BreakIterator`).*
> 2. *Similarity threshold filtering (discarding chunks below 0.65 cosine similarity).*
> 3. *Dynamic context budgeting and structured prompt guardrails.*
> 4. *Real-time RAG Triad evaluation metrics.*

Q2: Why did you use Neon PostgreSQL with `pgvector` instead of Pinecone, Chroma, or Milvus?

> ***Answer***:
> *Specialized vector databases cause **architectural fragmentation**—forcing the engineering team to maintain two separate databases: a relational database for user authentication, roles, and audit logs, and an external vector store for embeddings.*
> *Neon PostgreSQL with `pgvector` provides **unified ACID transactions**, relational foreign key joins (`ON DELETE CASCADE` between documents and chunks), zero synchronization lag, and sub-100ms retrieval speed—all within an industry-standard relational database engine.*

Q3: Explain how HNSW indexing works in `pgvector` and why it is superior to IVFFlat.

> ***Answer***:
> ***HNSW (Hierarchical Navigable Small World)** is a multi-layer graph-based Approximate Nearest Neighbor (ANN) index. Sparse upper layers provide long-range skip connections for coarse navigation, while dense lower layers provide fine-grained local clustering.*
> *Unlike **IVFFlat**, which clusters vectors into static Voronoi centroids and requires retraining whenever new documents are inserted, HNSW supports dynamic real-time inserts without retraining and maintains $O(\log N)$ search complexity with $>95\%$ recall.*

Q4: What is the mathematical definition of Cosine Similarity and how does PostgreSQL compute it?

> ***Answer***:
> *Cosine similarity measures the cosine of the angle between two multi-dimensional vectors: $$\text{Cosine Similarity} = \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}|_2 |\mathbf{v}|_2}$$ In PostgreSQL with `pgvector`, the `<=>` operator computes **Cosine Distance** ($1 - \text{Cosine Similarity}$). When vectors are normalized to unit length ($|\mathbf{u}| = 1$), the calculation reduces to simple dot products, which the CPU vectorizes efficiently using AVX-512 hardware acceleration.*

Q5: How do you prevent and measure hallucinations in your project?

> ***Answer***:
> *We enforce a three-tier defense:*
>
> 1. ***Prompt Guardrails**: The system prompt explicitly commands the model: "Answer strictly and solely using the provided context chunks. If the answer is not contained in the context, respond: 'The uploaded knowledge base does not contain sufficient information.' Never extrapolate."*
> 2. ***Similarity Threshold**: If retrieved vector similarity is below 0.65, we reject speculative generation.*
> 3. ***RAG Triad Faithfulness Metric**: We measure the ratio of generated claims that are logically entailed by retrieved source passages, achieving a verified **92.4% Faithfulness score** with a negligible 3.2% hallucination rate.*

Q6: Why did you choose Electron with React instead of Java Swing or JavaFX?

> ***Answer***:
> *While Java Swing and JavaFX are traditional GUI toolkits, they lack native streaming markdown renderers, syntax-highlighted code formatters, and responsive dark-mode styling frameworks like Tailwind CSS.*
> *By pairing a **Java Spring Boot backend** with an **Electron desktop wrapper**, we get the best of both worlds:*

> *enterprise Java compile-time performance and multithreading on the server, paired with modern desktop UX, native OS notifications, and streaming Server-Sent Events (SSE) in the client.*

Q7: What happens if an uploaded PDF is password-protected or corrupted?

> ***Answer***:
> *In Module 2 (`DocumentIngestionService`), Apache Tika detects encryption headers during MIME type analysis. If a file is password-protected or has malformed PDF streams, the asynchronous pipeline catches the `EncryptedDocumentException`, sets the document database status to `FAILED`, and logs the error in the audit log without crashing the Spring Boot thread pool or affecting active chat sessions.*